

EvoCause: LLM-Guided Evolution of Causal Graphs for Root Cause Analysis

Lei Zan¹, Keli Zhang¹, Shifeng Xie¹, Jiale Zheng¹, Zehao Xiao¹,
Zhiwei Dong¹, Ke Zhang¹, Ruichu Cai², Malik Tiomoko¹, Lujia Pan¹

¹Huawei Noah’s Ark Lab

²Guangdong University of Technology, Peng Cheng Laboratory
{xdzanlei, zxiao4ai, cairuichu}@gmail.com,
{zhangkeli1, zhengjiale2, zhangke200, malik.tiomoko, panlujia}@huawei.com,
shifeng.xie@telecom-paris.fr, zhiwei.dong@etu.sorbonne-universite.fr

Abstract

Modern telecommunication, cloud, and microservice systems emit correlated alarm cascades when components fail. Root cause analysis (RCA) aims to identify the small set of alarms that initiate each cascade. A common approach learns a causal graph from observational logs and predicts all zero-in-degree alarms in each incident-induced subgraph. However, the learned graph remains fixed and cannot benefit from expert diagnoses of historical incidents. We close this loop with **EvoCause**. Expert labels constrain which alarms should be source nodes but do not specify the edge edits needed to satisfy those constraints. EvoCause uses a large language model (LLM) to propose semantically plausible graph edits, while deterministic code validates node identities and acyclicity and retains the best graph on a labeled alignment set. At test time, the refined graph alone produces transparent predictions without an LLM call. We also release TeleRCA, an expert-annotated benchmark from a production telecommunication network containing 485,681 alarm events spanning 194 alarm types over 5,621 resources. On synthetic data, EvoCause initialized with the PC causal discovery algorithm outperforms the unrefined PC baseline, raising Node F1, Case EM, and Graph F1 by 11.59, 9.40, and 4.59 percentage points, respectively, while reducing nSHD by 0.2379. On TeleRCA, replacing human-readable alarm titles with anonymous identifiers lowers Node F1 and Case EM by 6.12 and 8.04 percentage points, respectively, indicating that alarm-name information contributes to graph refinement.

1 Introduction

Modern telecommunication, cloud, and microservice systems comprise many tightly coupled components, so a single fault can trigger a cascade of correlated alarms, known as an “alert storm” (Zhao et al. 2020). In this context, root cause analysis (RCA) is the task of identifying, among these alarms, the smallest set of actionable events whose remediation actually resolves the incident (Assaad, Ez-Zejjari, and Zan 2023). Rapid diagnosis is critical because major outages can disrupt many dependent services (O’Brien and Bajak 2021).

A common causal approach represents alarm types and their triggering relations as a directed acyclic graph (DAG) and predicts all zero-in-degree alarms in the subgraph induced by each incident. This framing provides traceable pre-

dictions and generalizes across incidents by encoding triggering relations rather than memorized cases.

Constructing such a graph manually is impractical at scale, so causal discovery algorithms learn it from observational data (Assaad, Devijver, and Gaussier 2022; Spirtes, Glymour, and Scheines 2000). However, their assumptions and dependence on clean data can produce spurious or missing edges in production alarm streams (Aït-Bachir et al. 2023).

Crucially, existing methods ignore a complementary source of information that could correct these structural errors, namely the historical incident logs curated by Site Reliability Engineering (SRE) teams, which pair observed alarm sequences with expert-labeled root causes. These diagnostic labels are sparse and expensive to obtain. More importantly, they provide task-level constraints rather than direct edge supervision: they can rule out graphs whose incident-induced source nodes disagree with expert diagnoses, but they generally do not identify a unique ground-truth DAG.

This feedback is indirect and underdetermined because expert labels specify which alarms should be sources but not which edge additions, removals, or reversals should enforce them without degrading predictions for overlapping incidents. Language models are useful because they can jointly interpret constraints from multiple incidents, the current graph, optimization history, and, when available, information from alarm titles. We therefore propose **EvoCause** (Evolve Causal Graph), which uses an LLM to rank a small set of semantically plausible edits from this combinatorial space rather than to predict the root cause directly. Deterministic procedures validate every edit, reject cyclic candidates, and retain the graph with the best alignment-set performance. The graph is the only optimized state, whereas its derived topological order is used only as prompt context rather than as a prediction rule. We evaluate the contribution of alarm-title information on TeleRCA by replacing only the titles supplied to the LLM with fixed anonymous identifiers while preserving the graph structure and incident data. Our contributions are summarized as follows:

- On the methodological side, we formulate incident-level expert feedback as *source-node constraints* on incident-induced subgraphs and characterize the corresponding set of root-cause-consistent causal graphs. Based on this formulation, we propose **EvoCause**, a discovery-agnostic causal graph refinement framework that integrates LLM-

guided semantic graph editing with deterministic structural validation and supervision-driven graph selection. EvoCause can refine graphs produced by different causal discovery algorithms without modifying their internal learning procedures, while ensuring that the resulting graphs remain structurally valid and consistent with expert RCA annotations.

- On the data side, we release **TeleRCA**, a large-scale, expert-labeled RCA benchmark derived from a real-world production telecommunication network. The dataset comprises 10,922 incidents, 485,681 alarm events, and 194 alarm types across thousands of network resources. Its combination of production alarm sequences and incident-level expert root-cause annotations provides a complementary testbed for advancing and evaluating event-sequence-based RCA methods.
- On the experimental side, we evaluate both RCA (Node F1 and Case EM) and graph reconstruction (Graph F1 and nSHD) on ten synthetic DAGs, together with RCA performance on TeleRCA. EvoCause improves all three discovery backbones. A controlled name-anonymization ablation on TeleRCA evaluates the contribution of human-readable alarm-title information, while additional backbone comparisons on synthetic data assess sensitivity to the choice of LLM.

2 Related Work

Causal graphs and expert feedback. Observational data generally identify a causal DAG only up to a Markov equivalence class under some standard assumptions. A completed partially directed acyclic graph (CPDAG) represents this class. Directed edges are shared by all compatible DAGs, while undirected edges remain ambiguous (Chickering 2002). Consistent edge-orientation background knowledge can orient some ambiguous edges and narrow the class (Fang et al. 2025). Propagating all implied orientations yields a maximally oriented partially directed acyclic graph (MPDAG), which represents the DAGs consistent with both the data and the background knowledge (Perković 2020; Guo and Perković 2022). Expert root-cause labels instead constrain source sets in incident-induced subgraphs and may require skeleton changes, so EvoCause searches a root-cause-consistent graph set that can span multiple equivalence classes. Interventional equivalence would apply only if incidents were known interventions (Hauser and Bühlmann 2012). In telecommunication RCA, Zhang et al. (2021) combine Hawkes-process discovery, conditional-independence tests, propagation embeddings, and influence maximization. CCCM (Zhang et al. 2024) introduces cluster-aware discovery for base-station alarms, while TTCTH (Li et al. 2025) combines topological-temporal representation learning with a Hawkes-process objective. For microservices, CIRCA (Ikram et al. 2022) treats failures as interventions, RUN (Lin et al. 2024) combines neural Granger causality with contrastive learning, and CausIL (Chakraborty et al. 2023) incorporates domain knowledge into instance-level causal graphs. For threshold-based IT systems, Zan et al. (2024) combine offline causal discovery with on-

line subgraph traversal. APGNN (Jiang and Bai 2023), Chain-of-Event (Yao et al. 2024), and Groot (Wang et al. 2021) construct event or alarm graphs for diagnosis using learned associations, interpretable edge parameters, or operator rules. These methods provide graph-learning and localization mechanisms but generally do not repeatedly revise an alarm-type DAG using accumulated expert root sets. HRLHF (Wang et al. 2023) does query engineers during dependency-graph discovery, whereas EvoCause uses previously labeled incidents as an offline task signal for refining a graph produced by any discovery backbone.

LLM-based RCA and graph refinement. LLM-based RCA systems have been studied for recommending root causes and mitigations from cloud-incident text (Ahmed et al. 2023). RCACopilot (Chen et al. 2024) combines incident-specific diagnostic collection with classification and explanation. RAgent (Wang et al. 2024) and ReAct-based systems (Roy et al. 2024) use tools and retrieval, while OpenRCA (Xu et al. 2025) and Flow-of-Action (Pei et al. 2025) provide agent-oriented evaluation or procedure-constrained diagnosis. These approaches mainly predict, explain, or plan actions for an individual incident, so their outputs need not update a shared causal model. Cloud Atlas (Xie et al. 2024) is closer to reusable graph construction because it synthesizes causal graphs from documentation, telemetry, and deployment feedback and then validates them with data. Outside RCA, CAMA refines a mathematical causal graph using question-answer feedback (Zan et al. 2026). EvoCause instead refines an externally discovered alarm graph using expert root-cause labels and performs test-time inference with the refined graph alone. The LLM proposes semantic candidates, while deterministic procedures validate node identities and acyclicity, select the best graph, and perform all test-time inference.

Datasets. The telecom data used by Zhang et al. (2021) provide root-cause labels for 6,000 sampled alarm transactions, with causal-edge annotations covering 15 of 78 alarm types. THPs (Cai et al. 2024) evaluates event-sequence structure learning rather than incident-level root localization. OpenRCA contains 335 software failures and more than 68 GB of logs, metrics, and traces (Xu et al. 2025). In comparison, TeleRCA targets alarm-sequence RCA and provides expert root-cause annotations for 10,922 production incidents. It does not provide a ground-truth propagation graph, so we use it to evaluate RCA rather than exact graph reconstruction.

3 EvoCause

This section formulates RCA on alarm cascades and presents EvoCause, which has learning and inference stages. Section 3.1 defines the problem and assumptions, Section 3.2 analyzes identifiability under root-cause feedback, Section 3.3 summarizes the framework, Section 3.4 describes graph construction and label-feedback alignment, and Section 3.5 explains root cause inference.

3.1 Problem Setup

Dataset Description. We consider a telecommunication network that generates alarms from a finite set of alarm types, denoted by $\mathbf{A}$. The dataset contains m incident sequences, $\mathcal{S} = \{\mathbf{s}_i\}_{i=1}^m$, collected over a global observation window $\mathbf{T}$. Each incident sequence $\mathbf{s}_i$ corresponds to a single incident and is represented as

$$\mathbf{s}_i = \{(a_j, t_j, l_j)\}_{j=1}^{n_i} \quad (1)$$

where n_i denotes the number of alarm events in incident $\mathbf{s}_i$. Each triplet represents one alarm event: $a_j \in \mathbf{A}$ represents the alarm type, $t_j \in \mathbf{T}$ represents its timestamp, and $l_j \in \{0, 1\}$ is an expert-provided annotation indicating whether the event is a root cause of the corresponding incident. Specifically, $l_j = 1$ indicates a root-cause event, whereas $l_j = 0$ indicates a non-root-cause event. The corresponding incident-level expert root set is $y_i = \{a_j : (a_j, t_j, l_j) \in \mathbf{s}_i, l_j = 1\}$, where repeated occurrences of the same alarm type are included only once. We further assume that the triggering relations among alarm types in $\mathbf{A}$ remain invariant throughout the observation window.

Alarm propagation graph. We represent the triggering relations among the alarm types $\mathbf{A}$ with a directed acyclic graph $\mathcal{G} = (\mathbf{A}, \mathbf{E})$, which we call an *alarm propagation graph* (APG). Its nodes are alarm types, and an edge $(a_i, a_j) \in \mathbf{E}$ means that an occurrence of a_i may trigger a_j . For an incident, RCA restricts $\mathcal{G}$ to the observed alarm types and predicts every zero-in-degree node in the induced subgraph as a root cause. The graph is the only persistent structural state. When a linear representation is useful for prompting, we deterministically derive a topological order $O(\mathcal{G}) = \text{TOPOLOGICALORDER}(\mathcal{G})$, in which a_i precedes a_j for every edge $(a_i, a_j) \in \mathbf{E}$. This order provides a compact linear view of the graph for the LLM. It is derived from $\mathcal{G}$ and is neither optimized nor used for root-cause prediction.

Incident-level source operator. Let $V_i \subseteq \mathbf{A}$ denote the distinct alarm types observed in incident $\mathbf{s}_i$. For a candidate APG $\mathcal{G}$, define

$$\text{Src}_{\mathcal{G}}(V_i) = \{v \in V_i : \text{Pa}_{\mathcal{G}}(v) \cap V_i = \emptyset\}. \quad (2)$$

EvoCause predicts the complete set $\text{Src}_{\mathcal{G}}(V_i)$ and neither ranks these nodes nor uses the ground-truth number of roots.

Learning Objective. A conventional causal approach obtains such a graph from the statistical regularities of the observed alarm events alone, without using the expert labels. A causal discovery algorithm $\text{CAUSALDISCOVERY}(\cdot)$ maps the alarm types and their timestamps to a DAG,

$$\hat{\mathcal{G}} = \text{CAUSALDISCOVERY}(\{(a_j, t_j)\}), \quad (3)$$

and root causes are then localized by a procedure $h(\cdot)$ that assigns a binary label to every event,

$$[\hat{l}_1, \dots, \hat{l}_{n_i}] = h(\hat{\mathcal{G}}, \{(a_j, t_j)\}_{j=1}^{n_i}). \quad (4)$$

In this pipeline, the expert labels l_j never enter the construction of $\hat{\mathcal{G}}$, because the graph is recovered from event statistics alone.

The core objective is to use expert-annotated incidents to refine the graph so that it recovers complete root sets more accurately. We split the incidents into an offline training set $\mathcal{S}_{\text{off}}$ and a held-out test set $\mathcal{S}_{\text{on}}$. For causal discovery, let $\pi_{\text{obs}}(\mathbf{s}_i) = \{(a_j, t_j)\}_{j=1}^{n_i}$ denote the label-free projection of incident $\mathbf{s}_i$, and define $\mathcal{D}_{\text{hist}} = \{\pi_{\text{obs}}(\mathbf{s}_i) : \mathbf{s}_i \in \mathcal{S}_{\text{off}}\}$. The initial graph is learned from $\mathcal{D}_{\text{hist}}$, while the labeled alignment set $\mathcal{D}_{\text{align}}$ is used for graph refinement and model selection. All candidate-graph decisions use $\mathcal{D}_{\text{align}}$, and $\mathcal{S}_{\text{on}}$ is evaluated only once after selection. We select

$$\mathcal{G}^* = \arg \max_{\mathcal{G}} \mathbb{E}_{(\mathbf{s}_i, y_i) \sim \mathcal{D}_{\text{align}}} [\mathbf{1}\{\text{RCAENGINE}(\mathbf{s}_i, \mathcal{G}) = y_i\}], \quad (5)$$

where y_i is the complete expert root set and $\mathbf{1}\{\cdot\}$ is the indicator function.

Root-cause-consistent graph set. Let $\mathcal{D}_L = \{(V_i, y_i)\}_{i=1}^m$ denote a set of labeled incidents, where $y_i \subseteq V_i$ is the expert-provided root set for incident i . We define the *root-cause-consistent* (RCC) graph set as

$$\mathcal{C}_{\text{RCC}}(\mathcal{D}_L) = \{\mathcal{G} \in \text{DAG}(\mathbf{A}) : \text{Src}_{\mathcal{G}}(V_i) = y_i, \forall i\}. \quad (6)$$

More generally, two graphs are RCA-equivalent on $\mathcal{D}_L$ if they produce the same source set for every labeled incident. Unlike a Markov equivalence class, the RCC set may contain graphs with different skeletons and conditional-independence structures. When exact consistency cannot be achieved because of label noise, a restricted search space, or optimization limitations, we quantify inconsistency using the empirical disagreement

$$\mathcal{L}_{\text{RCA}}(\mathcal{G}, \mathcal{D}_L) = \frac{1}{m} \sum_{i=1}^m \mathbf{1}\{\text{Src}_{\mathcal{G}}(V_i) \neq y_i\}. \quad (7)$$

The objective in (5) then selects the visited graph with the smallest empirical disagreement, using Node F1 to break ties.

Assumptions. Let $\mathcal{K}$ denote the resource-time contexts and let $\mathcal{G}^{(c)}$ be the alarm-type causal graph in context c .

Assumption 1 (Causal consistency) A shared DAG $\mathcal{G}^\dagger$ satisfies $\mathcal{G}^{(c)} = \mathcal{G}^\dagger$ for every $c \in \mathcal{K}$, corresponding to causal stationarity across contexts (Assaad, Devijver, and Gaussier 2022).

Assumption 2 (Causal sufficiency) No latent alarm event is a common cause of two observed alarm types (Spirtes, Glymour, and Scheines 2000; Assaad, Devijver, and Gaussier 2022).

Assumption 3 (Expert-label consistency) Every alignment label satisfies $y_i = \text{Src}_{\mathcal{G}^\dagger}(V_i)$.

These assumptions make the feedback well defined but do not make the RCC set unique. Violations of Assumptions 1 or 2 make the APG a task-effective summary rather than the unique data-generating graph. Let $\tilde{\mathcal{D}}_L = \{(V_i, \tilde{y}_i)\}_{i=1}^m$ denote a corrupted version of $\mathcal{D}_L = \{(V_i, y_i)\}_{i=1}^m$, where $\tilde{y}_i \neq y_i$ for at most q incidents. Then, for any fixed graph,

$$|\mathcal{L}_{\text{RCA}}(\mathcal{G}, \tilde{\mathcal{D}}_L) - \mathcal{L}_{\text{RCA}}(\mathcal{G}, \mathcal{D}_L)| \leq \frac{q}{m}. \quad (8)$$

Each proposal considers a batch of incidents, and candidate graphs are scored on the full alignment set. A few isolated errors therefore have bounded direct influence, although systematic errors can still alter the proposal path and selected graph. EvoCause consequently targets an RCA-effective graph with low empirical disagreement rather than unique DAG recovery.

3.2 Identifiability under Root-Cause Feedback

Why root labels do not generally identify one DAG. For every labeled root $r \in y_i$, consistency requires $\text{Pa}_{\mathcal{G}}(r) \cap V_i = \emptyset$. For every labeled non-root $v \in V_i \setminus y_i$, however, it only requires $\text{Pa}_{\mathcal{G}}(v) \cap V_i \neq \emptyset$. The latter is a disjunctive constraint: at least one observed parent must exist, but the label does not identify which parent is responsible. Root-cause feedback therefore generally leaves multiple admissible DAGs.

Proposition 1 (Monotone refinement). Let $\mathcal{C}_m$ be the RCC set induced by the first m labeled incidents. Then

$$\mathcal{C}_{m+1} \subseteq \mathcal{C}_m. \quad (9)$$

Indeed, the $(m+1)$ -st label adds one source-set constraint, giving $\mathcal{C}_{m+1} = \mathcal{C}_m \cap \{\mathcal{G} : \text{Src}_{\mathcal{G}}(V_{m+1}) = y_{m+1}\}$. Thus additional consistent feedback can only eliminate candidate graphs.

Proposition 2 (Training-set RCA invariance). All graphs in $\mathcal{C}_{\text{RCC}}(\mathcal{D}_L)$ return the same expert root set on every incident in $\mathcal{D}_L$, although they may disagree on unseen incidents and may contain different edge sets. Consequently, training-label consistency alone does not determine which admissible graph will generalize best.

A sufficient condition for uniqueness. Suppose that the true APG is a DAG, its undirected skeleton is known, expert labels are noiseless, and *root-separating coverage* holds: for every adjacency $\{u, v\}$ in the skeleton, the labeled set contains an edge-isolating incident whose relevant alarm-type set is $V_i = \{u, v\}$. For such an incident, the root set is $\{u\}$ if and only if the edge is $u \rightarrow v$, and it is $\{v\}$ if and only if the edge is $v \rightarrow u$. Every skeleton edge is therefore uniquely oriented, so the RCC set restricted to the known skeleton is a singleton. This condition is sufficient but intentionally strong and need not hold in TeleRCA. Without it, EvoCause targets task-level identification by selecting an RCA-effective representative rather than claiming recovery of the unique data-generating DAG.

3.3 Overview of EvoCause

EvoCause comprises a *learning stage* and an *inference stage* (Figure 1). In the learning stage, EvoCause first builds an initial alarm propagation graph $\mathcal{G}_0$ from historical alarm sequences using off-the-shelf causal discovery, and then refines it by aligning it with the downstream RCA task through LLM-based label feedback. In the inference stage, a new incident is resolved by running a transparent graph-based procedure on the refined graph $\mathcal{G}^*$.

3.4 Learning Stage

The learning stage has two components, namely (1) constructing an initial alarm propagation graph from historical sequences, and (2) aligning it with the RCA task through LLM-based label feedback.

Stage I: Initial APG construction. The first stage constructs an initial graph from the label-free historical alarm sequences $\mathcal{D}_{\text{hist}}$:

$$\mathcal{G}_0 = \text{CAUSALDISCOVERY}(\mathcal{D}_{\text{hist}}). \quad (10)$$

Here $\text{CAUSALDISCOVERY}(\cdot)$ is any algorithm that returns a DAG over alarm types. We instantiate it with PC (Spirtes, Glymour, and Scheines 2000), NOTEARS (Zheng et al. 2018), and THPs (Cai et al. 2024), which have different inductive biases. EvoCause treats discovery as a black box. At each refinement round, a deterministic topological order $O(\mathcal{G}) = \text{TOPOLOGICALORDER}(\mathcal{G})$ is derived from the current DAG solely to give the LLM a compact linearization of causal flow. It is not a second model state, and root-cause prediction depends only on zero in-degree in the incident-induced subgraph.

Stage II: APG alignment via label feedback. The second stage refines $\mathcal{G}_0$ into $\mathcal{G}^*$ using labeled alignment cases $\mathcal{D}_{\text{align}} = \{(\mathbf{s}_i, y_i)\}_{i=1}^N$, where $\mathbf{s}_i$ is an incident sequence and $y_i \subseteq V_i$ is its complete expert root set. Following the objective in (5), EvoCause searches for a graph that maximizes Case EM on the alignment set.

To optimize (5), we adopt a batch-based iterative optimization procedure over N_e epochs. At the start of epoch e , $\mathcal{D}_{\text{align}}$ is randomly permuted to obtain $\mathcal{D}_{\text{align}}^{(e)}$ and then partitioned into batches of size N_b . Each batch triggers one refinement round with four steps.

Step 1: Batch processing. A batch $\mathbf{B}_k$ of N_b labeled cases is drawn from the current epoch’s partition. Batching amortizes the cost of LLM invocation and allows each refinement round to aggregate feedback across multiple incidents, reducing sensitivity to any single case.

Step 2: Predict with the current graph. For every case $(\mathbf{s}_i, y_i) \in \mathbf{B}_k$ we run a graph-based RCA procedure on the current graph,

$$\hat{y}_i = \text{RCAENGINE}(\mathbf{s}_i, \mathcal{G}), \quad (11)$$

which restricts $\mathcal{G}$ to the alarm types observed in $\mathbf{s}_i$ and returns all zero-in-degree nodes of the incident-induced subgraph. The complete set of such source alarm types is predicted, and the ground-truth number of roots is never provided to the procedure. This yields a per-case triple $(\mathbf{s}_i, \hat{y}_i, y_i)$ whose mismatches are the discrepancies that the next step must explain.

Step 3: LLM proposes graph edits. We aggregate the per-case triples into a batch-level evidence set

$$\mathbf{E} = \{(\mathbf{s}_i, \hat{y}_i, y_i)\}_{i \in \mathbf{B}_k}, \quad (12)$$

and present $\mathbf{E}$, together with the current graph $\mathcal{G}$, its derived order $O(\mathcal{G})$, and an optimization history $\mathbf{H}$ (defined below), to a large language model $\mathcal{M}$ through an update prompt p_u . The LLM proposes only targeted graph edits

$$\Delta \mathcal{G} = \mathcal{M}(\mathcal{G}, O(\mathcal{G}), \mathbf{E}, \mathbf{H}, p_u), \quad (13)$$

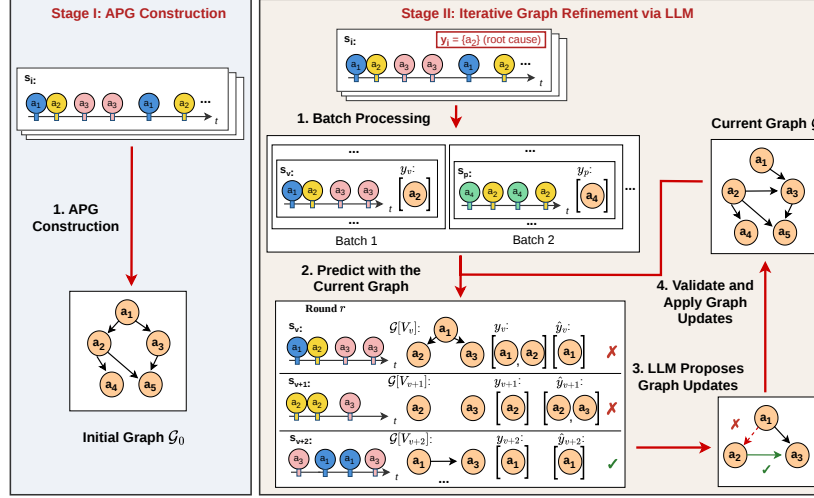


Figure 1: Overview of EvoCause. Stage I learns an initial alarm propagation graph $\mathcal{G}_0$ from historical alarm sequences. In Stage II, the current graph predicts the complete zero-in-degree source set for each expert-labeled incident in a batch. Prediction–label mismatches guide the LLM to propose edge additions, removals, or reversals. Deterministic code validates node identities and acyclicity, applies valid edits, and retains the graph with the best performance on the alignment set. Test-time RCA uses only the selected graph.

where $\Delta\mathcal{G}$ is a set of directed-edge edits using the operations $\{add, remove, reverse\}$. The LLM response contains only graph updates. The derived topological order $O(\mathcal{G})$ can neither be edited nor returned.

The LLM is a proposal model, not a causal oracle or a constraint solver. A labeled source-set error creates a disjunctive constraint: several edits may repair the current incident, yet those edits can have different effects on other incidents that share alarm types. Exhaustively testing all directed additions, removals, and reversals is quadratic in the number of alarm types per round before multi-edit combinations are considered, while a purely random proposer ignores both alarm semantics and cross-incident evidence. EvoCause uses the LLM to compress this search into a small set of proposals by jointly interpreting human-readable alarm identities, source-set mismatches, the current graph, its derived order, and prior outcomes. Structural validity and empirical utility remain determined by code and labeled data. Meaningful node names are not required because EvoCause can operate on arbitrary identifiers by interpreting source-set mismatches, graph structure, derived order, and prior outcomes. Human-readable alarm titles provide an additional signal when available.

The optimization history retains the last $K_{\mathcal{H}}$ rounds,

$$\mathbf{H} = \{(\mathcal{G}_{o-1}, S_{o-1}), \dots, (\mathcal{G}_{o-K_{\mathcal{H}}}, S_{o-K_{\mathcal{H}}})\}. \quad (14)$$

where $S_{o-1} = (L_{o-1}, F_{o-1})$ is the lexicographic score of graph $\mathcal{G}_{o-1}$ on the full alignment set $\mathcal{D}_{\text{align}}$. Here, L_{o-1} is Case EM and F_{o-1} is Node F1, which is used only to break ties. Specifically,

$$L_{o-1} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\text{RCAENGINE}(s_i, \mathcal{G}_{o-1}) = y_i\}. \quad (15)$$

Thus, model selection first maximizes exact root-set recovery over the full alignment set and uses partial node-level recovery only when two candidate graphs have equal Case EM.

Step 4: Validate and apply edits. The parser first checks that every operation is in $\{add, remove, reverse\}$ and that both endpoints belong to $\mathbf{A}$. Valid edits are applied to obtain a tentative graph

$$\mathcal{G}' = \text{APPLYGRAPHUPDATES}(\mathcal{G}, \Delta\mathcal{G}). \quad (16)$$

The tentative graph is accepted if and only if $\mathcal{G}'$ remains a DAG. Otherwise, the complete proposed update is rejected and the current graph is retained. Any topological order needed by the next prompt is then recomputed deterministically from the accepted graph, so no graph order synchronization state or compatibility check is required.

These checks guarantee structural feasibility, not causal correctness. The refinement loop may visit several structurally valid candidate graphs. Following each round, EvoCause evaluates the current graph on the labeled alignment set and stores the graph with the highest Case EM, using Node F1 only as a tie-breaker.

Termination. The four steps repeat across batches and epochs until either all N_e epochs are processed or the round counter reaches the budget N_r . To improve efficiency, the procedure also terminates early if the graph remains unchanged for c_{stop} consecutive batches. The graph with the highest Case EM on the alignment set is returned as $\mathcal{G}^*$ and used for all downstream RCA. The full procedure is summarized in Appendix A.1.

3.5 Inference Stage

At test time, a new incident $s_{\text{new}} \in \mathcal{S}_{\text{on}}$ is resolved using the refined graph $\mathcal{G}^*$ alone. The procedure identifies the alarm

types present in s_{new} , restricts $\mathcal{G}^*$ to the subgraph they induce, and returns the complete set of nodes whose in-degree in this induced subgraph is zero. These source alarm types constitute the predicted root-cause set. Formally,

$$\hat{y}_{\text{new}} = \text{RCAENGINE}(s_{\text{new}}, \mathcal{G}^*). \quad (17)$$

This is the same procedure used to score candidate graphs during learning, so the criterion optimized offline is exactly the one applied online. No LLM call is made at inference time. Every prediction therefore traces back to specific edges in $\mathcal{G}^*$, and every such edge is attributable either to the initial discovery stage or to a recorded LLM graph edit during refinement, providing an audit trail that operators can inspect. Expert labels are used only during offline alignment to score candidate graphs and construct mismatch evidence. Their effect is retained in $\mathcal{G}^*$, so held-out incidents require neither labels nor LLM calls.

4 Experiments

We evaluate EvoCause on synthetic networks with known causal graphs and on TeleRCA, an expert-labeled production telecommunication dataset. Synthetic experiments assess RCA, graph reconstruction, and backbone sensitivity using non-semantic node identifiers, while TeleRCA evaluates RCA, robustness to causal-discovery initialization, and the contribution of human-readable alarm titles.

4.1 Experimental Setup

For both datasets, we use non-overlapping 80% training and 20% test splits. Each initial graph is learned only from the training split. Within that split, 10% of the incidents form the labeled alignment set used for graph refinement and model selection. The held-out test split is evaluated once after the best alignment-set graph has been selected.

Evaluation metrics. We evaluate root cause analysis on both datasets using Node F1 and Case EM. Let $\hat{y}_i$ and y_i denote the predicted and true root cause sets for test incident i , and let N be the number of test incidents. Node F1 measures micro-averaged node-level overlap, while Case EM, where EM denotes exact match, measures incident-level accuracy:

$$\text{Node F1} = \frac{2 \sum_{i=1}^N |\hat{y}_i \cap y_i|}{\sum_{i=1}^N (|\hat{y}_i| + |y_i|)}, \quad \text{Case EM} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i]. \quad (18)$$

Node F1 gives partial credit for recovering part of the true root set, whereas Case EM counts an incident as correct only when the complete predicted and true sets are identical. On synthetic data, the known ground-truth graph also allows evaluation of graph reconstruction. Let $\hat{E}$ and E denote the estimated and true directed edge sets:

$$\text{Graph F1} = \frac{2|\hat{E} \cap E|}{|\hat{E}| + |E|}, \quad \text{nSHD} = \frac{\text{SHD}(\hat{G}, G)}{|E|}. \quad (19)$$

Graph F1 measures directed-edge recovery. Here, $\text{SHD}(\hat{G}, G) = N_{\text{add}} + N_{\text{del}} + N_{\text{rev}}$, where the three terms are the minimum numbers of unit-cost edge additions, deletions,

and reversals required to transform $\hat{G}$ into G . Higher Node F1, Case EM, and Graph F1 are better, while lower nSHD is better. Because TeleRCA has no ground-truth propagation graph, graph-reconstruction metrics are reported only on synthetic data.

Baselines and controlled variants. Across both datasets, we compare unrefined graphs learned by PC, NOTEARS, and THPs with Chain-of-Event (Yao et al. 2024), CCCM (Zhang et al. 2024), APGNN (Jiang and Bai 2023), and RUN (Lin et al. 2024), using the released default settings for the latter four methods. For PC and NOTEARS, we follow the alarm-transaction representation of Zhang et al. (2021). Random uses a DAG with matched node and edge counts on synthetic data and a random alarm ordering on TeleRCA. EvoCause uses Qwen3-32B with alignment fraction $\rho = 0.1$, batch size $N_b = 20$, epoch count $N_e = 10$, round budget $N_r = 500$, history window $K_H = 5$, and retry budget $N_{\text{retry}} = 3$ per batch. Hyperparameter robustness and label-feedback efficiency are examined in Appendices A.5 and A.6, respectively. On synthetic data, two Qwen3-4B variants assess backbone sensitivity only (Yang et al. 2025). On TeleRCA, EvoCause-PC (Anonymous IDs) replaces alarm titles with fixed anonymous identifiers while keeping all other settings unchanged to test the contribution of human-readable titles.

4.2 Synthetic Data

Dataset. Synthetic data allow evaluation against fully known graphs. We generate ten independent Erdős–Rényi DAGs, each with 50 alarm-type nodes and 70 directed edges before transitive reduction. The reduction preserves all nodes but removes shortcut edges. We retain 2,000 valid incidents per graph, with one to five root alarms per incident. Each active edge generates one offspring event after an exponential delay whose edge-specific mean is sampled once from $[30, 300]$ seconds and fixed across incidents. We report the mean and standard deviation across the ten graphs. Because alarm types use numeric identifiers, these experiments evaluate structural refinement without alarm-name semantics.

Analysis. Table 1 shows that label-feedback refinement improves RCA for all three initial discovery methods, so the benefit is not tied to the inductive bias of PC. With PC initialization, EvoCause raises Node F1 and Case EM by 11.59 and 9.40 percentage points, increases Graph F1 by 4.59 percentage points, and reduces nSHD by 0.2379. RCA improves more strongly than complete edge recovery for NOTEARS and THPs, which is consistent with expert labels directly constraining incident-level source sets but only indirectly constraining the complete edge set. EvoCause-PC gives the strongest joint RCA and graph-reconstruction result, showing that refinement preserves the value of a strong initial graph. Because synthetic nodes use arbitrary numeric identifiers, these improvements also show that EvoCause can exploit source-set mismatches and graph context without relying on lexical information. The backbone comparisons evaluate sensitivity to the choice of LLM and demonstrate that the framework can operate with different LLM backbones.

Method	Node F1 (%) $\uparrow$	Case EM (%) $\uparrow$	Graph F1 (%) $\uparrow$	nSHD $\downarrow$
Random	47.23 $\pm$ 8.42	6.05 $\pm$ 3.18	5.71 $\pm$ 4.89	1.8714 $\pm$ 0.1421
PC	52.03 $\pm$ 6.17	17.80 $\pm$ 4.48	64.80 $\pm$ 21.58	0.7859 $\pm$ 0.7483
NOTEARS	30.03 $\pm$ 2.95	7.43 $\pm$ 2.38	34.38 $\pm$ 11.29	0.8593 $\pm$ 0.1554
THPs	37.05 $\pm$ 6.54	10.60 $\pm$ 3.48	47.93 $\pm$ 4.12	0.6882 $\pm$ 0.0317
Chain-of-Event (Yao et al. 2024)	53.45 $\pm$ 8.11	22.98 $\pm$ 6.29	53.71 $\pm$ 4.92	1.4795 $\pm$ 0.2141
CCCM (Zhang et al. 2024)	58.10 $\pm$ 7.11	25.43 $\pm$ 5.51	46.23 $\pm$ 6.98	1.6179 $\pm$ 0.3218
APGNN (Jiang and Bai 2023)	61.13 $\pm$ 7.60	24.17 $\pm$ 6.25	51.99 $\pm$ 3.94	1.5741 $\pm$ 0.1745
RUN (Lin et al. 2024)	60.80 $\pm$ 4.69	23.30 $\pm$ 4.30	54.67 $\pm$ 8.55	1.2931 $\pm$ 0.2995
EvoCause-PC (4B-Inst.)	57.78 $\pm$ 6.03	21.92 $\pm$ 5.41	66.61 $\pm$ 14.86	0.6887 $\pm$ 0.5529
EvoCause-PC (4B-Think.)	62.80 $\pm$ 6.72	26.62 $\pm$ 6.98	68.07 $\pm$ 10.63	0.5673 $\pm$ 0.2100
EvoCause-PC (32B)	63.62 $\pm$ 6.61	27.20 $\pm$ 6.51	69.39 $\pm$ 8.76	0.5480 $\pm$ 0.1462
EvoCause-NOTEARS (32B)	55.09 $\pm$ 6.40	20.95 $\pm$ 4.68	37.94 $\pm$ 8.57	0.7837 $\pm$ 0.0645
EvoCause-THPs (32B)	55.74 $\pm$ 7.24	20.75 $\pm$ 4.41	50.32 $\pm$ 3.20	0.6645 $\pm$ 0.0283

Table 1: Synthetic-data results on ten transitively reduced Erdős–Rényi DAGs. Values are mean $\pm$ standard deviation across graph instances. 32B, 4B-Inst., and 4B-Think. denote Qwen3-32B, Qwen3-4B-Instruct-2507, and Qwen3-4B-Thinking-2507. Full component and backbone analyses are reported in Appendices A.3 and A.4.

4.3 Real-World Telecommunication Data

Dataset. TeleRCA contains 10,922 incidents comprising 485,681 alarm events across 119,039 timestamps, 5,621 network resources, and 194 alarm types, collected from a production telecommunication network in Jakarta, Indonesia, between March and July 2025. Potentially sensitive information was anonymized to meet privacy and publication requirements. The incident-file structure and field definitions are provided in Appendix A.2.

Method	Node F1 (%) $\uparrow$	Case EM (%) $\uparrow$
Random	71.69 $\pm$ 2.07	62.15 $\pm$ 2.79
PC	65.18	53.98
NOTEARS	70.69	61.03
THPs	76.59	68.86
CCCM	75.95	67.35
Chain-of-Event	65.82	54.81
APGNN	84.14 $\pm$ 0.91	78.30 $\pm$ 1.23
RUN	57.65 $\pm$ 0.03	45.03 $\pm$ 0.05
EvoCause-PC	92.58 $\pm$ 0.04	89.61 $\pm$ 0.05
EvoCause-PC (Anonymous IDs)	86.46 $\pm$ 0.06	81.57 $\pm$ 0.08
EvoCause-NOTEARS	92.47 $\pm$ 0.07	89.44 $\pm$ 0.10
EvoCause-THPs	92.48 $\pm$ 0.04	89.46 $\pm$ 0.05

Table 2: RCA performance on TeleRCA. Values are mean $\pm$ standard deviation over five runs where available. EvoCause-PC (Anonymous IDs) replaces the original alarm titles supplied to the LLM with fixed anonymous identifiers. All EvoCause variants use Qwen3-32B, whereas deterministic discovery baselines are single runs.

Analysis. Table 2 shows that EvoCause reaches nearly identical RCA performance from PC, NOTEARS, and THPs despite substantial differences among their unrefined results. With PC initialization, EvoCause improves Node F1 from 65.18% to 92.58% and Case EM from 53.98% to 89.61%, gains of 27.40 and 35.63 percentage points. This convergence indicates that label-feedback alignment reduces dependence on the initial discovery bias. Replacing the original alarm

titles with anonymous identifiers lowers Node F1 to 86.46% and Case EM to 81.57%, reductions of 6.12 and 8.04 percentage points. Because all other settings remain unchanged, this result indicates that human-readable alarm-name information provides additional guidance for graph-edit proposals. The anonymized variant still exceeds the strongest non-EvoCause baseline by 2.32 percentage points in Node F1 and 3.27 percentage points in Case EM, showing that feedback-guided structural refinement remains effective without lexical information. Since TeleRCA has no ground-truth propagation graph, these results demonstrate diagnostic utility and robustness rather than exact structural recovery.

5 Conclusion

We release TeleRCA, an expert-annotated production benchmark, and propose EvoCause, a discovery-agnostic framework that uses historical expert diagnoses to refine alarm graphs. Because incident labels specify desired source nodes but not the required edge changes, an LLM proposes semantically plausible graph updates. Deterministic procedures validate node identities and acyclicity and retain the best graph on a labeled alignment set. Experiments on synthetic data and TeleRCA show that EvoCause improves root cause analysis over unrefined discovery graphs and other RCA baselines, while synthetic results also show improved graph reconstruction. The TeleRCA name-anonymization result further indicates that human-readable alarm-title information provides additional guidance for refinement. At inference, the refined graph alone produces predictions without expert labels or LLM calls. Since root-cause labels generally identify a root-cause-consistent graph set rather than a unique data-generating DAG, future work will strengthen identifiability by incorporating intervention records, temporal evidence, and expert edge-level constraints. We also plan to model resource-specific and time-varying propagation, account for latent common causes, and improve robustness to noisy expert labels through active selection of informative incidents.

References

- Ahmed, T.; Ghosh, S.; Bansal, C.; Zimmermann, T.; Zhang, X.; and Rajmohan, S. 2023. Recommending root-cause and mitigation steps for cloud incidents using large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 1737–1749.
- Assaad, C. K.; Devijver, E.; and Gaussier, E. 2022. Survey and Evaluation of Causal Discovery Methods for Time Series. *Journal of Artificial Intelligence Research*, 73: 767–819.
- Assaad, C. K.; Ez-Zejjari, I.; and Zan, L. 2023. Root Cause Identification for Collective Anomalies in Time Series given an Acyclic Summary Causal Graph with Loops. In Ruiz, F.; Dy, J.; and van de Meent, J.-W., eds., *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, 8395–8404. PMLR.
- Ait-Bachir, A.; Assaad, C. K.; de Bignicourt, C.; Devijver, E.; Ferreira, S.; Gaussier, E.; Mohanna, H.; and Zan, L. 2023. Case Studies of Causal Discovery from IT Monitoring Time Series. *arXiv:2307.15678*.
- Cai, R.; Wu, S.; Qiao, J.; Hao, Z.; Zhang, K.; and Zhang, X. 2024. THPs: Topological Hawkes Processes for Learning Causal Structure on Event Sequences. *IEEE Transactions on Neural Networks and Learning Systems*, 35(1): 479–493.
- Chakraborty, S.; Garg, S.; Agarwal, S.; Chauhan, A.; and Saini, S. K. 2023. CausIL: Causal graph for instance-level microservice data. In *Proceedings of the ACM Web Conference (WWW)*, 2905–2915.
- Chen, Y.; Xie, H.; Ma, M.; Kang, Y.; Gao, X.; Shi, L.; Cao, Y.; Gao, X.; Fan, H.; Wen, M.; Zeng, J.; Ghosh, S.; Zhang, X.; Zhang, C.; Lin, Q.; Rajmohan, S.; Zhang, D.; and Xu, T. 2024. Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys)*, 674–688.
- Chickering, D. M. 2002. Learning Equivalence Classes of Bayesian-Network Structures. *Journal of Machine Learning Research*, 2: 445–498.
- DeepSeek-AI; Guo, D.; Yang, D.; et al. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*.
- Fang, Z.; Zhao, R.; Liu, Y.; and He, Y. 2025. On the Representation of Pairwise Causal Background Knowledge and Its Applications in Causal Inference. *Journal of Machine Learning Research*, 26(229): 1–73.
- Guo, F. R.; and Perković, E. 2022. Efficient Least Squares for Estimating Total Effects under Linearity and Causal Sufficiency. *Journal of Machine Learning Research*, 23(104): 1–41.
- Hauser, A.; and Bühlmann, P. 2012. Characterization and Greedy Learning of Interventional Markov Equivalence Classes of Directed Acyclic Graphs. *Journal of Machine Learning Research*, 13(79): 2409–2464.
- Ikram, A.; Chakraborty, S.; Mitra, S.; Saini, S.; Bagchi, S.; and Kocaoglu, M. 2022. Root cause analysis of failures in microservices through causal discovery. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, 31158–31170.
- Jiang, W.; and Bai, Y. 2023. APGNN: Alarm Propagation Graph Neural Network for fault detection and alarm root cause analysis. *Computer Networks*, 220: 109485.
- Li, Y.; Kong, Y.; Yin, S.; and Li, J. 2025. Topological-Temporal Convolution Transformer Hawkes Process for Causal Structural Learning in Telecom Networks. *IEEE Internet of Things Journal*, 12(17): 36019–36033.
- Lin, C.-M.; Chang, C.; Wang, W.-Y.; Wang, K.-D.; and Peng, W.-C. 2024. Root cause analysis in microservice using neural Granger causal discovery. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 206–213.
- O’Brien, M.; and Bajak, F. 2021. Why did Amazon Web Services crash? Here’s what it means. <https://globalnews.ca/news/8434673/why-amazon-web-services-crash/>. Accessed: 2024-07-25.
- Pei, C.; Wang, Z.; Liu, F.; Li, Z.; Liu, Y.; He, X.; Kang, R.; Zhang, T.; Chen, J.; Li, J.; Xie, G.; and Pei, D. 2025. Flow-of-Action: SOP-enhanced LLM-based multi-agent system for root cause analysis. In *Companion Proceedings of the ACM Web Conference (WWW)*.
- Perković, E. 2020. Identifying Causal Effects in Maximally Oriented Partially Directed Acyclic Graphs. In *Proceedings of the 36th Conference on Uncertainty in Artificial Intelligence*, volume 124, 530–539. PMLR.
- Roy, D.; Zhang, X.; Bhavre, R.; Bansal, C.; Las-Casas, P. H. B.; Fonseca, R.; and Rajmohan, S. 2024. Exploring LLM-based agents for root cause analysis. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, 208–219.
- Spirtes, P.; Glymour, C. N.; and Scheines, R. 2000. *Causation, Prediction, and Search*. MIT Press, 2nd edition.
- Wang, H.; Wu, Z.; Jiang, H.; Huang, Y.; Wang, J.; Köprü, S.; and Xie, T. 2021. Groot: An event-graph-based approach for root cause analysis in industrial settings. In *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 419–429.
- Wang, L.; Zhang, C.; Ding, R.; Xu, Y.; Chen, Q.; Zou, W.; Chen, Q.; Zhang, M.; Gao, X.; Fan, H.; Rajmohan, S.; Lin, Q.; and Zhang, D. 2023. Root cause analysis for microservice systems via hierarchical reinforcement learning from human feedback. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 5116–5125.
- Wang, Z.; Liu, Z.; Zhang, Y.; Zhong, A.; Wang, J.; Yin, F.; Fan, L.; Wu, L.; and Wen, Q. 2024. RCAGENT: Cloud root cause analysis by autonomous agents with tool-augmented large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*, 4966–4974.
- Xie, Z.; Zheng, Y.; Ottens, L.; Zhang, K.; Kozyrakis, C.; and Mace, J. 2024. Cloud Atlas: Efficient fault localization for cloud systems using language models and causal insight. *arXiv preprint arXiv:2407.08694*.

- Xu, J.; Zhang, Q.; Zhong, Z.; He, S.; Zhang, C.; Lin, Q.; Pei, D.; He, P.; Zhang, D.; and Zhang, Q. 2025. OpenRCA: Can large language models locate the root cause of software failures? In *The Thirteenth International Conference on Learning Representations*.
- Yang, A.; Li, A.; Yang, B.; et al. 2025. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*.
- Yao, Z.; Pei, C.; Chen, W.; Wang, H.; Su, L.; Jiang, H.; Xie, Z.; Nie, X.; and Pei, D. 2024. Chain-of-Event: Interpretable root cause analysis for microservices through automatically learning weighted event causal graph. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE Companion)*, 50–61.
- Zan, L.; Assaad, C. K.; Devijver, E.; Gaussier, E.; and Aït-Bachir, A. 2024. On the Fly Detection of Root Causes from Observed Data with Application to IT Systems. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 5062–5069. ACM.
- Zan, L.; Zhang, K.; Cai, R.; and Pan, L. 2026. CAMA: Enhancing mathematical reasoning in large language models with causal knowledge. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 34566–34574.
- Zhang, K.; Kalander, M.; Zhou, M.; Zhang, X.; and Ye, J. 2021. An Influence-Based Approach for Root Cause Alarm Discovery in Telecom Networks. In *Service-Oriented Computing – ICSOC 2020 Workshops*, 124–136. Springer.
- Zhang, K.; Zhu, X.; Zheng, J.; Cao, J.; Zhang, K.; and Pan, L. 2024. Cluster-Aware Causal Discovery Framework for Root Cause Analysis of Base Station Alarms. In *2024 IEEE Globecom Workshops (GC Wkshps)*, 1–6. IEEE.
- Zhao, N.; Chen, J.; Peng, X.; Wang, H.; Wu, X.; Zhang, Y.; Chen, Z.; Zheng, X.; Nie, X.; Wang, G.; et al. 2020. Understanding and handling alert storm for online service systems. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice*, 162–171.
- Zheng, X.; Aragam, B.; Ravikumar, P. K.; and Xing, E. P. 2018. DAGs with NO TEARS: Continuous Optimization for Structure Learning. In *Advances in Neural Information Processing Systems*, volume 31, 9492–9503.

A Appendix

A.1 EvoCause Optimization Procedure

Algorithm 1 compares candidate graphs using the lexicographic score $S = (\text{CaseEM}, \text{NodeF1})$, prioritizing exact incident resolution and using partial node recovery only to break ties. At epoch e , $\mathcal{D}_{\text{align}}^{(e)}$ is a random permutation of $\mathcal{D}_{\text{align}}$ used to form the batches. The graph is the only persistent structural state, while its topological order is recomputed for each prompt. The LLM returns only graph updates, invalid or cyclic candidates are rejected, and the graph with the best alignment-set score is retained.

Algorithm 1: EvoCause: LLM-Guided Evolution of Causal Graphs for RCA

Require: Historical sequences $\mathcal{D}_{\text{hist}}$, labeled alignment cases $\mathcal{D}_{\text{align}} = \{(s_i, y_i)\}_{i=1}^N$, discovery algorithm CAUSALDISCOVERY, LLM $\mathcal{M}$, prompt p_u , batch size N_b , epochs N_e , round budget N_r , patience c_{stop} , seed ξ

Ensure: Refined graph $\mathcal{G}^*$

```

1:  $\mathcal{G} \leftarrow \text{CAUSALDISCOVERY}(\mathcal{D}_{\text{hist}})$ 
2:  $S^* \leftarrow \text{Score}(\mathcal{D}_{\text{align}}, \mathcal{G})$   $\mathcal{G}^* \leftarrow \mathcal{G}$ 
3:  $r \leftarrow 0$   $c \leftarrow 0$   $\mathcal{H} \leftarrow \emptyset$ 
4: for  $e = 1$  to  $N_e$  do
5:    $\mathcal{D}_{\text{align}}^{(e)} \leftarrow \text{Shuffle}(\mathcal{D}_{\text{align}}; \xi + e)$ 
6:    $\{\mathcal{B}_k\}_{k=1}^K \leftarrow \text{Partition}(\mathcal{D}_{\text{align}}^{(e)}, N_b)$ 
7:   for  $k = 1$  to  $K$  do
8:      $\mathcal{G}_{\text{prev}} \leftarrow \mathcal{G}$ 
9:      $\mathcal{E} \leftarrow \{(s_i, \text{RCAENGINE}(s_i, \mathcal{G}), y_i) : (s_i, y_i) \in \mathcal{B}_k\}$ 
10:     $O(\mathcal{G}) \leftarrow \text{TOPOLOGICALORDER}(\mathcal{G})$ 
11:     $\Delta\mathcal{G} \leftarrow \mathcal{M}(\mathcal{G}, O(\mathcal{G}), \mathcal{E}, \mathcal{H}, p_u)$ 
12:     $\mathcal{G}' \leftarrow \text{APPLYGRAPHUPDATES}(\mathcal{G}, \Delta\mathcal{G})$ 
13:    if  $\text{ValidUpdates}(\Delta\mathcal{G}, \mathbf{A})$  and  $\text{IsDAG}(\mathcal{G}')$  then
14:       $\mathcal{G} \leftarrow \mathcal{G}'$ 
15:    else
16:      Reject the update and keep  $\mathcal{G}$  unchanged
17:    end if
18:     $S \leftarrow \text{Score}(\mathcal{D}_{\text{align}}, \mathcal{G})$ 
19:    if  $S >_{\text{lex}} S^*$  then
20:       $S^* \leftarrow S$   $\mathcal{G}^* \leftarrow \mathcal{G}$ 
21:    end if
22:     $c \leftarrow c + 1$  if  $\mathcal{G} = \mathcal{G}_{\text{prev}}$ , otherwise  $c \leftarrow 0$ 
23:    Append  $(\mathcal{G}, S)$  to  $\mathcal{H}$ , then set  $r \leftarrow r + 1$ 
24:    if  $r \geq N_r$  or  $c \geq c_{\text{stop}}$  then
25:      return  $\mathcal{G}^*$ 
26:    end if
27:  end for
28: end for
29: return  $\mathcal{G}^*$ 

```

A.2 TeleRCA Data Structure

Each TeleRCA incident is stored as one JSON file with two main fields: `nodes` contains the resources and alarms observed during the incident, and `causeNode` contains the expert annotation. Each alarm record includes an identifier, a human-readable title, and a UTC timestamp. The resource

identifier is retained in the released record but is not part of the event triplet (a_j, t_j, l_j) used in the method formulation.

Each key in `causeNode` identifies a root-cause resource and maps to one or more alarm identifiers. These identifiers are matched to alarm records under the corresponding entry in `nodes`. The matched alarm titles define the expert root set, and the remaining alarms are downstream symptoms. The abbreviated example below contains one labeled `Physical Port Down` root alarm.

```

1  {"nodes": [
2    {"device": "JKT-GATG-OPT-H910D",
3     "alarms": [{"id": "a1", "title": "BFD session down", "t": "03:22:15"}]},
4    {"device": "JKT-RJG-AN1-H8X08",
5     "alarms": [{"id": "a2", "title": "Physical Port Down", "t": "03:22:04"},
6                {"id": "a3", "title": "Link Down", "t": "03:22:04"}]},
7  "causeNode": {"JKT-RJG-AN1-H8X08": ["a2"]}]

```

A.3 Synthetic Component Ablations

Table 3 reports the component-ablation results. We initialize EvoCause with PC, use Qwen3-32B as the default proposal model, and change one prompt component at a time. w/o HISTORY removes prior graph–score pairs, while w/o ORDER removes the topological linearization derived from the current graph. These variants evaluate the contribution of structural prompt context on synthetic graphs whose nodes have arbitrary numeric identifiers.

Removing the derived order causes the larger degradation, lowering Node F1, Case EM, and Graph F1 by 2.90, 5.01, and 0.95 percentage points, respectively, while increasing nSHD by 0.0751. Removing optimization history causes smaller reductions of 0.81, 0.87, and 0.70 percentage points in Node F1, Case EM, and Graph F1, respectively, while increasing nSHD by 0.0266. These results show that both structural context components improve refinement, with the derived topological order having the larger overall contribution.

A.4 LLM Backbone Sensitivity

We fix the initial PC graph, prompt, alignment cases, graph-edit budget, and deterministic validator, and vary only the LLM backbone. We compare Qwen3-4B-Instruct-2507, Qwen3-4B-Thinking-2507, DeepSeek-R1-Distill-Llama-70B, and the default Qwen3-32B (Yang et al. 2025; DeepSeek-AI et al. 2025). Because these models may differ in several respects beyond parameter count, this experiment is treated as a backbone sensitivity analysis rather than an isolation of any specific model capability.

Among the backbone results in Table 3, Qwen3-32B obtains the best mean result on all four metrics, while performance varies across the remaining backbones. Because the models differ in multiple respects, this comparison does not isolate the cause of the observed differences. We therefore use it only to assess the sensitivity of EvoCause to the selected LLM backbone.

Variant or backbone	Node F1 (%) $\uparrow$	Case EM (%) $\uparrow$	Graph F1 (%) $\uparrow$	nSHD $\downarrow$
<i>Component ablations</i>				
w/o Topological Order Context	60.72 $\pm$ 6.84	22.19 $\pm$ 5.56	68.44 $\pm$ 16.02	0.6231 $\pm$ 0.4029
w/o Optimization History	62.81 $\pm$ 6.46	26.33 $\pm$ 6.87	68.69 $\pm$ 12.20	0.5746 $\pm$ 0.2558
EvoCause	63.62 $\pm$ 6.61	27.20 $\pm$ 6.51	69.39 $\pm$ 8.76	0.5480 $\pm$ 0.1462
<i>LLM backbone sensitivity</i>				
Qwen3-4B-Instruct-2507	57.78 $\pm$ 6.03	21.92 $\pm$ 5.41	66.61 $\pm$ 14.86	0.6887 $\pm$ 0.5529
DeepSeek-R1-Distill-Llama-70B	61.18 $\pm$ 6.63	25.53 $\pm$ 6.46	67.70 $\pm$ 12.26	0.5918 $\pm$ 0.3591
Qwen3-4B-Thinking-2507	62.80 $\pm$ 6.72	26.62 $\pm$ 6.98	68.07 $\pm$ 10.63	0.5673 $\pm$ 0.2100
Qwen3-32B	63.62 $\pm$ 6.61	27.20 $\pm$ 6.51	69.39 $\pm$ 8.76	0.5480 $\pm$ 0.1462

Table 3: Component ablations and LLM-backbone sensitivity on the ten synthetic graph instances with PC initialization. The component study uses Qwen3-32B except for the stated variant. Values are mean $\pm$ standard deviation.

A.5 Hyperparameter Robustness

Figure 2 summarizes the sensitivity of EvoCause to the round budget, epoch count, and batch size. On TeleRCA, incidents are partitioned 4:1 into non-overlapping train and test sets. The default values are $N_r = 500$ refinement rounds, $N_e = 10$ epochs, and $N_b = 20$ cases per batch. We vary one parameter at a time, run each setting five times, and report Node F1 and Case EM.

Round budget. For this robustness study, we use 20% of the training split as the labeled alignment set, i.e., $\rho_{\text{rob}} = 0.2$, and vary the round budget over $N_r \in \{100, 300, 500, 700, 900\}$. Performance improves up to approximately $N_r = 700$ and then stabilizes, while variability generally decreases as the budget increases.

Epoch count. We vary $N_e \in \{5, 7, 10, 13, 15\}$. Both metrics remain within a narrow range, with a mild peak near the default $N_e = 10$, indicating diminishing returns after a moderate number of shuffled passes through the alignment cases.

Batch size. We vary $N_b \in \{10, 20, 30, 40, 50\}$. The default $N_b = 20$ gives the best mean performance. Larger batches slightly degrade both metrics and increase variance, consistent with the greater number of cross-incident constraints that must be reconciled in one proposal.

A.6 Label-Feedback Efficiency

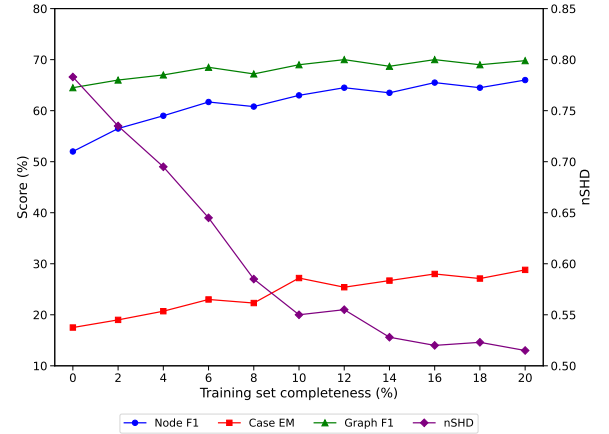


Figure 3: Synthetic-data performance under PC initialization as the labeled alignment fraction increases from 0% to 20%. The x-axis label “training set completeness” denotes the percentage of the training split used for label-feedback alignment.

Figure 3 varies the percentage of the training split used as the labeled alignment set from 0% to 20% in steps of two percentage points under PC initialization. The 0% point is the unrefined PC graph. All four metrics improve sharply once a small labeled subset is introduced. Node F1, Case EM, and Graph F1 then fluctuate within a comparatively narrow range, whereas nSHD continues to decline as more labels constrain the admissible graph set. The default 10% setting lies in this stable region and balances annotation cost with refinement quality.

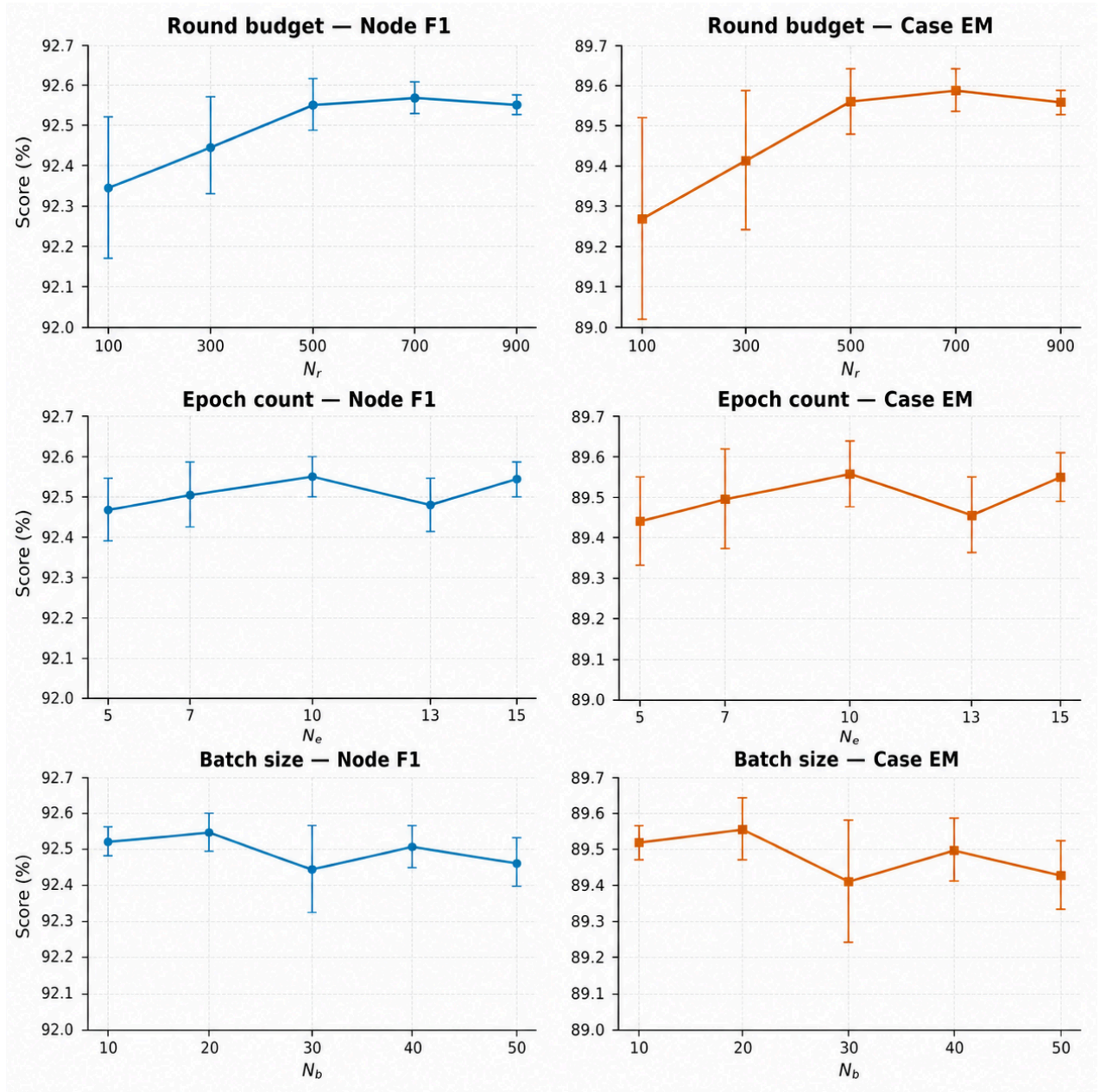


Figure 2: Sensitivity to the refinement-round budget, epoch count, and batch size. Each point reports mean and standard deviation over five runs. The left and right columns show Node F1 and Case EM, respectively.